1. Linear

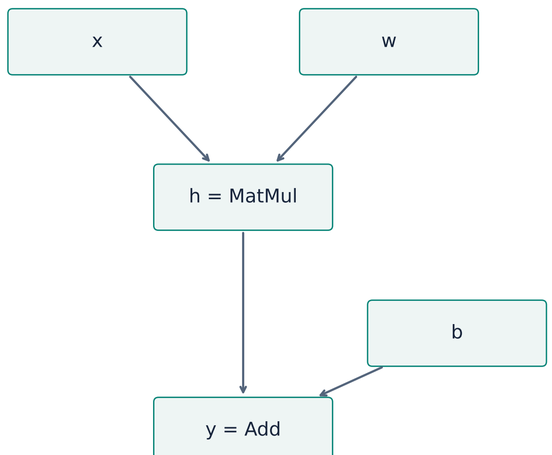
y = x @ w + b

	Jac: explicit nodes + edges	Python: FX node construction

1 Multiply	`h = MatMul();`
	`x +>: A() :+> h;`
	`w +>: B() :+> h;`
		`h = g.call_function(mm, (x, w))`

2 Add bias	`y = Add();`
	`h +>: A() :+> y;`
	`b +>: B() :+> y;`
		`y = g.call_function(add, (h, b))`

Computation graph



Shared h is computed once.

Semantic diagram; not a literal Jac or FX node export.

Aligned by operation semantics, not by source lines. Presentation excerpts only; runnable sources are unchanged.

Jac shorthand: A/B/X stand for typed input-port edges. Names, values, node/walker definitions and execution are omitted.

FX shorthand: mm/add/relu are ATen targets; graph inputs, GraphModule wrapping and execution are omitted.

Full sources and verification: jac/examples/graph_frontend_comparison/ | No timing or usability claim.

2. Shared computation + residual

h = x @ w; y = relu(h) + h

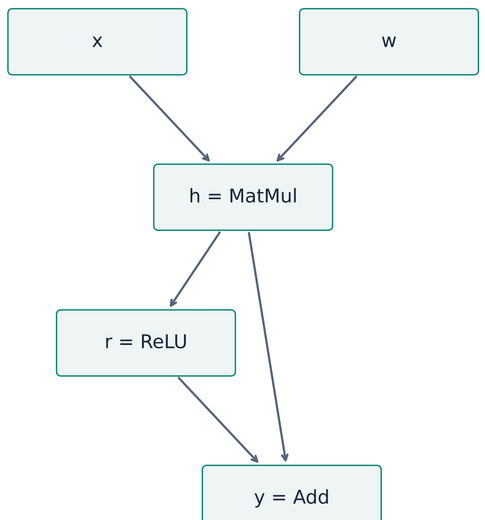
	Jac: explicit nodes + edges	Python: FX node construction

1 Multiply once	`h = MatMul();`
	`x +>: A() :+> h;`
	`w +>: B() :+> h;`
		`h = g.call_function(mm, (x, w))`

2 ReLU	`r = ReLU();` `h +>: X() :+> r;`	`r = g.call_function(relu, (h,))`

3 Residual add	`y = Add();`
	`r +>: A() :+> y;`
	`h +>: B() :+> y;`
		`y = g.call_function(add, (r, h))`

Computation graph



Shared h is computed once.

Semantic diagram; not a literal Jac or FX node export.

Aligned by operation semantics, not by source lines. Presentation excerpts only; runnable sources are unchanged.

Jac shorthand: A/B/X stand for typed input-port edges. Names, values, node/walker definitions and execution are omitted.

FX shorthand: mm/add/relu are ATen targets; graph inputs, GraphModule wrapping and execution are omitted.

Full sources and verification: jac/examples/graph_frontend_comparison/ | No timing or usability claim.

3. Conditional: same result, different construction

h = x @ w; y = h + x if c else h - x

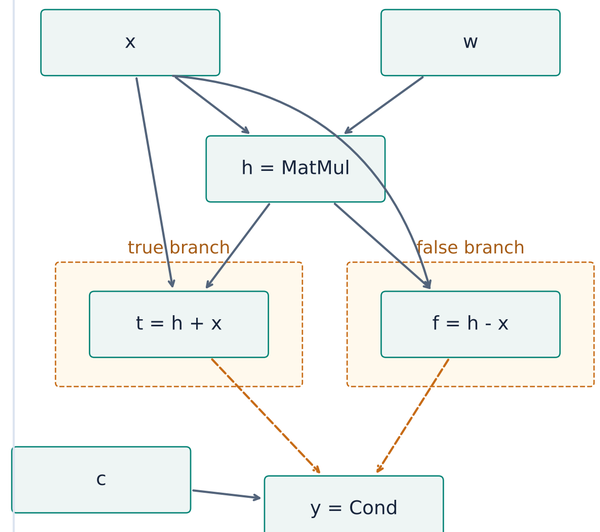
	Jac: explicit nodes + edges	Python: FX node construction

1 Shared multiply	`h = MatMul();`
	`x +>: A() :+> h;`
	`w +>: B() :+> h;`
		`h = g.call_function(mm, (x, w))`

2 Branch bodies	`t = Add(); f = Subtract();`
	`h +>: A() :+> t;`
	`x +>: B() :+> t;`
	`h +>: A() :+> f;`
	`x +>: B() :+> f;`
		`t, f = cond_modules(g, root, add, sub)`
		`# t(h,x) -> clone(h+x)`
		`# f(h,x) -> clone(h-x)`

3 Select + join	`gate +>: Then() :+> t;`
	`gate +>: Else() :+> f;`
	`t +>: X() :+> y;`
	`f +>: X() :+> y;`
		`r = g.call_function(cond, (c, t, f, (h, x)))`
		`y = g.call_function(getitem, (r, 0))`

Computation graph



Only one branch executes.
Dashed arrows: lazy result selection.

Semantic diagram; not a literal Jac or FX node export.

Aligned by operation semantics, not by source lines. Presentation excerpts only; runnable sources are unchanged.

Jac shorthand: A/B/X stand for typed input-port edges. Names, values, node/walker definitions and execution are omitted.

Cond is not a one-to-one node mapping: Jac uses a gate + control edges + output join; FX uses two branch modules.

Gate inputs c/h/x, branch enabling, routing and FX output wrapping are omitted; custom types are not Jac builtins.